

WEEK 4 • DAY 1

Unstructured Data Foundations

What's Inside a Document, and How Foundry Sees It

1-Hour Lecture Session — Concepts & Architecture (No Live Platform Demo)

Rajesh Pasham

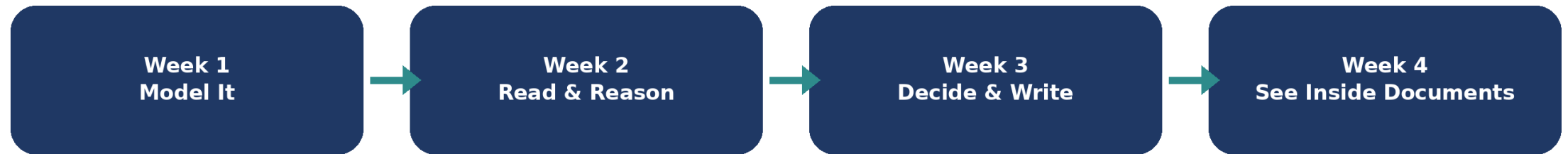
Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and architecture lecture — no live Palantir platform work today
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3)	Insurance Document Intelligence Pipeline — extracting and linking real claim, policy, and medical documents to your existing Ontology

Where Today Fits

From structured objects to the documents behind them



Every object you've built since Week 1 has a property like `fileReference` or a linked Claim Document — but until now, nothing in your workflows has actually looked at what's inside those files. Week 4 is about that gap: turning a PDF, a scanned form, or a photo into something your Ontology and your AI workflows can actually use.

Today's Roadmap

Three topic groups, building toward Wednesday's document pipeline

1 Foundry & Unstructured Data

What a media set is, the types of files it holds, and how to reference one without copying it

2 Making Documents Machine-Readable

Classification, text extraction (raw text vs. OCR), and metadata — turning a file into usable content

3 Finding What You Need

Content-based search, and how an extracted document connects back to your Ontology

By the End of Today

The vocabulary Wednesday's document pipeline lab assumes



Explain what a media set is, and why it's a first-class Foundry concept

Not just a folder of files — a governed, schema-aware collection with its own operations



Know the difference between raw text extraction and OCR

And why picking the wrong one silently produces bad results, not an error



Understand what metadata a document carries beyond its text

Page count, file name, format — useful on its own, before any AI is involved



See how an extracted document connects back to the Ontology objects you already built

The same Claim Document and Insurance Policy relationships from Week 1, now carrying real content

TOPIC 1

Foundry & Unstructured Data

What a media set is, the types of files it holds, and how to reference one without copying it

What Counts as Unstructured Data

Anything that doesn't fit neatly into rows and columns



Documents

Claim forms, policy PDFs, medical records, police reports — the paperwork behind every insurance process



Images

Photos of vehicle damage, property damage, scanned handwritten forms



Audio and video

Recorded claim calls, inspection walkthroughs — less common in this program, but the same underlying concept applies

Media Sets: A First-Class Concept

Not just a folder — a governed collection with a common schema

A Media Set Is

A collection of media files that share a common schema — for example, all PDFs, or all vehicle-damage photos

Why It's Not Just a Folder

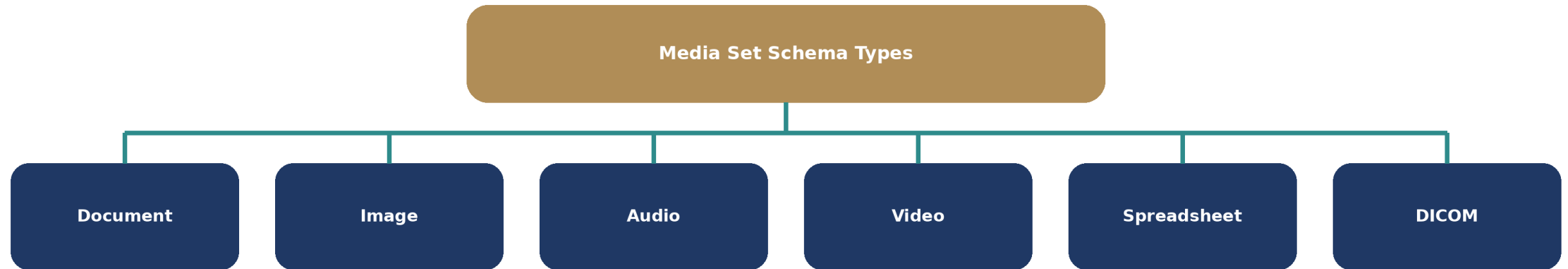
Media sets carry the same governance you've used since Week 1 — Markings, security policies, and versioned uploads — and support schema-specific operations directly, like OCR or format-based filtering

Schema Types Foundry Recognizes

Document, image, audio, video, spreadsheet, and DICOM (medical imaging) — each with its own available operations

Schema Types, at a Glance

Six recognized types — today's lab works primarily with Document and Image



Media References: Using Data Without Copying It

The same reference discipline, applied to files instead of objects



A media reference points to a file, rather than duplicating it

The same principle as a Link Type from Week 1 — you reference the relationship, not a second copy of the data



Media references can live as a column in an ordinary dataset

Associating a PDF with its file name, page count, and extracted text as additional columns — tabular data sitting alongside unstructured content



Media reference object properties bring this into the Ontology

This is what lets a Claim Document object actually display and preview its underlying file in Object Explorer or Workshop

TOPIC 2

Making Documents Machine-Readable

Classification, text extraction, and metadata — turning a file into usable content

Document Classification

Knowing what kind of file you have, before deciding what to do with it



Media sets can be filtered by type and format

For example, narrowing a mixed media set down to just PDFs before running a PDF-specific operation



Classification is what makes batch processing possible

A pipeline that handles claim forms, medical records, and photos together needs to know which is which before applying the right extraction method to each

Text Extraction: Raw Text vs. OCR

The single most important decision in this topic — and an easy one to get wrong silently

Raw Text Extraction

For born-digital PDFs — documents where the text is already embedded and selectable. Fast, and the extracted text is exactly what's in the file.

OCR (Optical Character Recognition)

For scanned documents and images with no embedded text layer — a photo of a form, a faxed document. OCR reads the visual characters and reconstructs text, which can introduce errors.

Why the Distinction Matters

Running raw text extraction on a scanned document returns nothing useful. Running OCR on a born-digital PDF works, but slower and with unnecessary risk of misread characters — picking correctly matters for both speed and accuracy.

Auto Mode: Letting the Platform Decide

A practical middle ground for mixed-quality document sets



Auto mode inspects each page individually

It determines, page by page, whether text can be extracted directly or whether OCR is required — useful when a single PDF mixes born-digital pages with scanned inserts



Auto mode isn't free of judgment calls — it's a default worth checking, not a guarantee

For a batch of claim documents where quality is inconsistent, Auto is usually the safer starting point; for a known, uniform document type, an explicit choice can be faster

Metadata Extraction

What a document tells you before you've read a single word of it



Every media item carries metadata beyond its content

File name, format, page count, upload date — available directly, without running any extraction at all

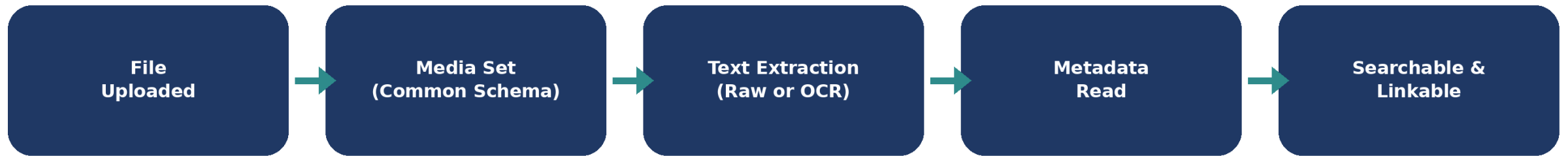


Metadata is often enough to route a document, before extraction even runs

A claim document's page count or format alone can flag it as incomplete or the wrong type, before you spend compute extracting its text

Putting It Together: From File to Usable Content

The path every document takes before it's ready for Week 4's lab



TOPIC 3

Finding What You Need

Content-based search, and how an extracted document connects back to your Ontology

Content-Based Search

Searching what's inside a document, not just its file name



Once text is extracted, it can be searched like any other text field

“Find every claim document mentioning water damage” becomes a real query, not a manual review task



This is the foundation Day 2 builds on

Content-based search here is exact-match — Day 2 introduces semantic and hybrid retrieval, which finds conceptually similar content even without matching words

Connecting Documents Back to the Ontology

The same object model you built in Week 1, now carrying real content

Your Claim Document object type has had a fileReference property since Week 1 — today's concepts are what finally make that property mean something. Wednesday's lab links extracted text and metadata from real claim forms and medical records back to the Claim and Insurance Policy objects you've been working with since Week 1, using the exact media reference object properties introduced in Topic 1.

Common Pitfalls

Patterns worth avoiding before Wednesday's lab

Assuming every PDF needs OCR

A born-digital PDF already has embedded text — running OCR on it wastes compute and risks introducing errors that raw extraction wouldn't have

Treating a media set like an ordinary dataset

Media sets have their own governance and operations — the same Markings and security discipline from Week 1 still apply, but the mechanics are specific to media

Skipping metadata because extraction seems more important

Metadata is often the cheapest, fastest way to classify or route a document, before spending compute on full text extraction

Forgetting that a media reference is a reference, not a copy

The same discipline from Week 1's Link Types — duplicating file content defeats the purpose and multiplies storage cost

Recap & What's Next

What We Covered Today

- Media sets, schema types, and media references
- Document classification
- Raw text extraction vs. OCR, and Auto mode
- Metadata extraction
- Content-based search and the link back to your Ontology

Day 2 — Document Pipelines & Semantic Search

- Chunking strategies and embedding models
- Vector search and hybrid (semantic + keyword) retrieval
- Retrieval quality evaluation, citation and provenance
- Also a concepts lecture — no live platform work



Questions

Rajesh Pasham